PROJET DE DATA MINING

december 2025

Contents

1 INTRODUCTION & MOTIVATION **2**

2 MÉTHODES D'APPRENTISSAGE SUPERVISÉS **2**
2.1 Régression Linéaire : Fondements, Hypothèses et résultats 2
2.2 Régression Logistique 3
2.3 arbre de décision 4
2.4 Le gradient Boosting 6

3 MÉTHODES D'APPRENTISSAGE NON SUPERVISÉS **8**
3.1 K-means clustering 8
3.2 Réduction de Dimensionnalité: Analyse en Composantes Principales (ACP) 10
3.3 DBSCAN : Clustering Basé sur la Densité 10
3.3.1 Fonctionnement de DBSCAN 11

4 MACHINE LEARNING SUR NOTRE BASE DE DONNÉES **11**
4.0.1 DESCRIPTION ET OBSERVATION DE LA BASE DE DONNÉES 11
4.0.2 Analyse exploratoire des données 12
4.0.3 Préparation de nos données pour le machine learning 18
4.0.4 Traitement du déséquilibre (suréchantillonage) 19
4.1 MODÈLE SUPERVISÉ: CLASSIFICATION 21
4.1.1 Classification avec une regression logistique 21
4.1.2 Classification avec un Gradient Boostiong(XGBoost) 23
4.1.3 classification d'un nouvel individu dans une des classes 24
4.1.4 Conclusion Partielle 25
4.2 MODÈLE NON SUPERVISÉ: K-MEANS 25
4.2.1 Visualisation des clusters 28
4.2.2 classification d'un nouvel individu dans un des clusters 31
4.2.3 Conclusion Partielle 32

5 COMPARAISON ENTRE LES DEUX TYPES DE MODÈLES **33**

6 CONCLUSION **33**

1 INTRODUCTION & MOTIVATION

L'**apprentissage automatique** (Machine Learning) représente aujourd'hui l'un des domaines les plus dynamiques et transformateurs de l'informatique moderne. Depuis les travaux pionniers d'**Arthur Samuel** dans les années 1950 qui définit l'apprentissage automatique comme " le domaine d'étude qui donne aux ordinateurs la capacité d'apprendre sans être explicitement programmés ", le champ a connu des avancées spectaculaires.

La classification des méthodes d'apprentissage en supervisé et non supervisé remonte aux travaux fondateurs de **Duda et Hart** qui ont établi les bases théoriques de ces deux paradigmes. Cette distinction fondamentale structure toujours la recherche contemporaine, même si des approches hybrides ont émergé pour combiner leurs avantages respectifs.

2 MÉTHODES D'APPRENTISSAGE SUPERVISÉS

Formellement, l'apprentissage supervisée consiste à apprendre une fonction $f : X \to Y$ à partir d'un ensemble d'exemples étiquetés $\{(x_i, y_i)\}_{i=1}^{n}$ où $xi_{\in}X$ sont les observations et $y_i \in Y$ sont les labels correspondants. L'objectif est de minimiser l'erreur de généralisation sur de nouvelles observations. Comme modèles supervisés en machine learning, on peut citer le modèle de régression linéaire, le modèle de régression logistiques, le modèle

2.1 Régression Linéaire : Fondements, Hypothèses et résultats

La régression linéaire, introduite par **Legendre** (1805) et **Gauss** (1809), représente l'une des méthodes statistiques les plus anciennes. Le modèle suppose une relation linéaire entre les variables explicatives $X \in \mathbb{R}^{n \times k}$ et la variable réponse $Y \in \mathbb{R}^n$. Le modèle est:

$$Y = X\beta + \epsilon \qquad \epsilon \sim \mathcal{N}(0, \sigma^2 \cdot I)$$

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + ... + \beta_{k-1} X_{(k-1)i} + \epsilon_i \quad i = 1, ..., n$$

où

- β est le vecteur des coefficients
- ϵ est le vecteur d'erreur

Les hypothèses d'un tel modèle sont:

- absence de multicolinéarité: les X_k sont linéairement indépendants
- les X_k sont connus $\forall k$
- on a $n \geq k$
- $\mathbb{E}(\epsilon) = \vec{0}$
- $\mathbb{E}(\epsilon\epsilon') = \sigma^2 I_n$

L' estimation des coefficients β se fait par la méthode des **moindres carrés ordinaires** qui minimise la somme des carrés des erreurs

$$SSE(\beta) = \sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^{n} (Y_i - X_i\beta)^2$$

En notation matricielle:

$$SSE(\beta) = (Y - X\beta)^T(Y - X\beta)$$

On va dériver cette expression par rapport à β afin de trouver son minimum:

$$\frac{\partial SSE}{\partial \beta} = -2X^T(Y - X\beta) = 0$$

On tire β et on obtient:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

- β_0:c'est l'ordonnée à l'origine(Valeur prédite lorsque tous les $X_j = 0$)
- c'est la variation moyenne de Y lorsque X_j augmente de une unité, avec les autres variables constantes

Lorsque tous nos hypothèses sont vérifiées, notre estimateur $\hat{\beta}$ verifie certaines propriétés:

- $\hat{\beta}$ est non biaisée:$\mathbb{E}[\hat{\beta}] = \beta$
- LA variance vaut: $\mathbb{V}(\hat{\beta}) = \sigma^2(X^T X)^{-1}$
- la prédiction recherchée pour un individu (x_0, Y_0) est $\hat{Y_0} = x_0\hat{\beta}$

2.2 Régression Logistique

Initialement développée par **Berkson** (1944) , la régression logistique est un modèle fondamental en statistiques et en machine learning pour prédire des probabilités d'événements binaires (0 ou 1). Plutôt que de prédire directement 0 ou 1, elle prédit la probabilité qu'un événement se produise.
la régression logistique postule que: Si on veut prédire $Y \in \{0, 1\}$, en connaissant $X = \}x_{i1}, x_{i2}, ..., x_{ip}\}$ alors $\mathbb{P}(Y = 1 \mid X) = p(X) = \frac{1}{1+e^{-(\beta_0+\beta_1 \cdot x_1 + ... + \beta_p \cdot x_p)}}$

On définit $\eta = \beta_0 + \beta_1 \cdot x_1 + ... + \beta_p \cdot x_p$ donc $\mathbb{P}(Y = 1 \mid X) = p(X) = \frac{1}{1+e^{-\eta}}$

Le but est toujours de minimiser l'erreur. Cette fois ci, nous allons appliquer la méthode du **maximum de vraisemblance**: Les Y_i suivent une loi de Bernoulli donc $f(y_i \mid X_i) = p_i^{y_i}(1 - y_i)^{1-y_i}$

La vraisemblance totale est donc

$$L(\beta) = \prod_{i=1}^{n} p_i^{y_i}(1 - y_i)^{1-y_i}$$

La log vraisemblance est:

$$l(\beta) = \sum_{i=1}^{n} [y_i log(p_i) + (1 - y_i) log(1 - p_i)]$$

avec $p_i = \sigma(\beta^T X_i)$

On cherche $\hat{\beta} = arg\max_\beta l(\beta)$, Pour cela, on calcule les dérivées partielles : $\frac{\partial l}{\partial \beta_j} = \sum_{i=1}^{n}(y_i - p_i)x_{ij}$
En notation vectorielle, on obtient:

$$\bigtriangledown l(\beta) = X^T(y - p)$$

Où :

- X=matrice n × p
- y=vecteur des réponses
- $p = (p_1, ..., p_n)$

Pour résoudre $\bigtriangledown l(\beta) = X^T(y-p)$, on doit avoir recours aux méthodes numériques. Nous allons appliquer la méthode de **NEWTON-RAPHSON** dont la formule itérative est:

$$\beta^{t+1} = \beta^t - H^{-1}(\beta^t) \bigtriangledown l(\beta^t)$$

où:

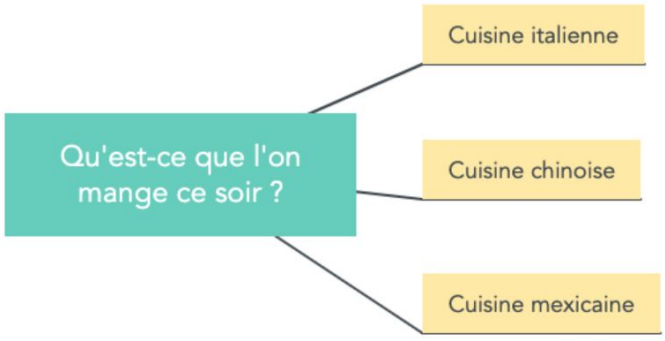
- H= hessienne$=-X^TWX$
- g=gradient $= X^T(y-p)$
- W=matrice diagonale $W_i = p_i(1-p_i)$

2.3 arbre de décision

Introduits par **Breiman et al**. en 1984, les arbres **CART**(Classification and Regression Trees) partitionnent récursivement l'espace des caractéristiques. Les **arbres de décision** permettent non seulement de présenter visuellement des informations mais aussi de les hiérarchiser.Cette méthode de décision est couramment utilisée dans le milieu informatique, médicale et scientifique pour explorer et exploiter des données au quotidien.

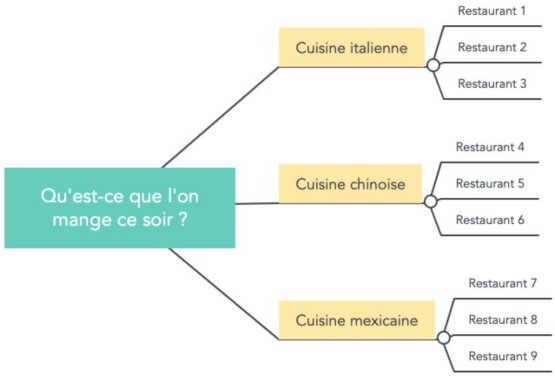
On distingue plusieurs parties sur l'arbre de décision :

- **La racine**: C'est de là que part toute notre réflexion. La racine c'est la décision à laquelle on fait face. C'est la question principale à laquelle on cherche à répondre. Un exemple simpliste de " racine" serait : ≪ Qu'est-ce que l'on mange ce soir ?≫
- **Les branches porteuses**: Les branches porteuses représentent nos principaux choix. Ce sont nos grandes options. Pour reprendre notre exemple, les branches porteuses pourraient être par exemple : Lorsque nos choix sont simples, les branches de l'arbre s'arrêtent ici. Mais lorsqu'ils sont plus

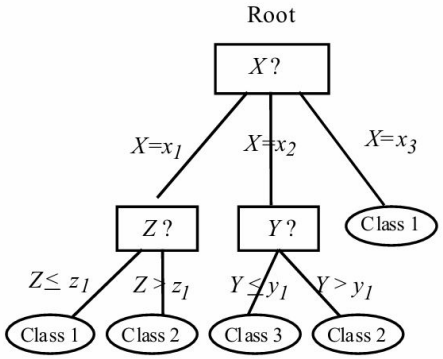


complexes, ils comportent aussi d'autres sous-branches.

- **Les sous-branches**: Les sous-branches sont des extensions de branches ou encore des sous-choix.



- **Les noeuds**: ce sont les points de rencontre de deux ou plusieurs branches. Chaque ≪nœud≫ intermédiaire réalise un test portant sur une variable dont le résultat indique la branche à suivre dans l'arbre. Pour classer un nouveau cas: suivre le chemin partant de la racine (nœud initial) à une feuille de l'arbre en effectuant les différents tests à chaque nœud.



Pour construire l'arbre, on cherche la meilleure variable et le meilleur seuil à chaque nœud pour diviser les données en sous-ensembles plus ≪ purs ≫

1. Dans le cas d'une **classification** , On utilise l'**impureté de Gini** définie par:

$$Gini(S) = 1 - \sum_{k=1}^{K} p_k^2$$

Où

- p_k est la proportion de la classe k dans l'ensemble S
- le **Gain de Gini** définit par:

$$\Delta Gini = Gini(S) - \frac{\mid S_L \mid}{\mid S \mid} Gini(S_L) - \frac{\mid S_R \mid}{\mid S \mid} Gini(S_R)$$

On choisit la variable et le seuil qui maximise cette valeur

On peut aussi utiliser le **critère d'entropie**: $Entropie(S) = -\sum_{k=1}^{K} p_k log_2 p_k$

2. Dans le cadre d'une régression on utilise plutôt la **variance**:

$$Var(S) = \frac{1}{\mid S \mid} \sum_{i \in S} (Y_i - \bar{Y}_s)^2$$

Le gain sera dans ce cas

$$\Delta Var = Var(S) - \frac{\mid S_L \mid}{\mid S \mid} Var(S_L) - \frac{\mid S_R \mid}{\mid S \mid} Var(S_R)$$

On va donc à gauche ou à droite en fonction de a direction qui minimise cette variance

Lorsqu'on utilise l'arbre de décision, on le parcourt depuis la racine en suivant les tests précédents sur chaque nœud. Arrivé à la feuille, on renvoie la classe majoritaire ou bien la moyenne selon si on veut faire une prédiction ou une classification

La **forêt d'arbre aléatoire**(Random Forest)est une méthode qui consiste à diviser nos données en n échantillons(On fait un échantillonnage par Bootstrap), pour chaque échantillon , faire une arbre de décision et déterminer les paramètres, et les observations. Le résultat final sera obtenu en faisant un dernier arbre de décision sur le nouvel ensemble formé des n observation précédentes, obtenues pour chaque échantillon

2.4 Le gradient Boosting

Le **Gradient Boosting** est aujourd'hui l'une des techniques les plus performantes pour traiter des données structurées. Son principe clé est l'**itération**: on cherche à améliorer la prédiction globale en entraînant séquentiellement de nouveaux modèles sur les résidus (les erreurs) des modèles précédents.Contrairement à d'autres méthodes qui cherchent à trouver le meilleur modèle unique tout de suite, le Gradient Boosting construit sa solution couche par couche. Voici par étape comment se déroule le processus logique :

1. On commence par un modèle simple (souvent très basique) qui fait une première prédiction. Naturellement, celle-ci est imparfaite et laisse un écart avec la réalité : c'est ce qu'on appelle le **résidu**.

2. Plutôt que de jeter ce premier modèle, on en entraîne un second. Mais l'objectif de ce second modèle n'est pas de prédire la cible originale, mais de prédire l'erreur commise par le premier. Il agit comme un **correcteur**.

3. Le modèle " amélioré " n'est pas simplement le second modèle, mais la somme du premier modèle et du correcteur

$$f_M(x) = f_0(x) + \sum_{m=1}^{M} \eta_m h_m(x)$$

Où
$h_m(x)$ est un modèle faible
η_m est le *learning rate*
M est le nombre d'étapes de Boosting

Dans des algorithmes comme **XGBoost**, ce cycle est répété des centaines, voire des milliers de fois. À chaque étape, on ajoute un nouveau "correcteur" qui se focalise sur les erreurs laissées par l'ensemble des modèles précédents.

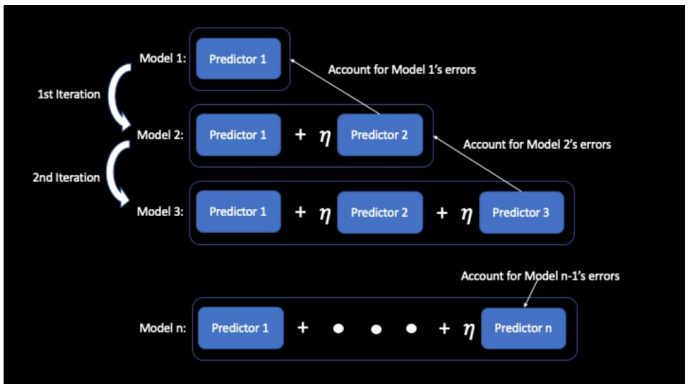
Mathématiquement, on dit que c'est un **modèle additif** : la prédiction finale est la somme de toutes les petites corrections apportées successivement. L'intuition derrière cette répétition est que si chaque

nouveau modèle parvient à capturer une partie de l'erreur restante, le résidu global va progressivement converger vers 0

Pour éviter le **surapprentissage** (overfitting), le gradient boosting adopte une stratégie contre-intuitive : il utilise volontairement des modèles rudimentaires, appelés ***Weak Learners***: Plutôt que d'utiliser un arbre de décision très complexe et profond capable de tout mémoriser, on contraint fortement la complexité des correcteurs (par exemple, en limitant la profondeur des arbres). L'objectif n'est pas que ce correcteur résolve tout le problème d'un coup, mais qu'il capture juste la tendance générale de l'erreur, sans apprendre le bruit spécifique aux données.

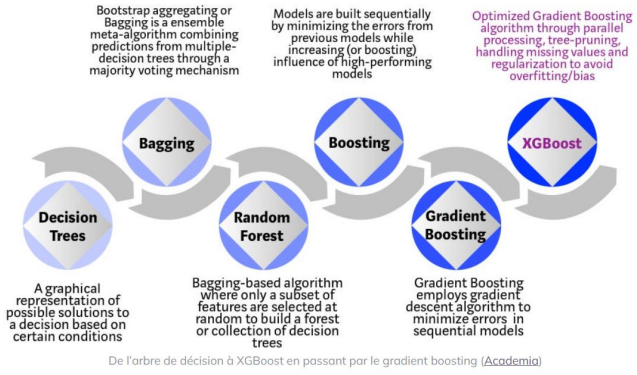
Pour renforcer cette prudence, on utilise un second mécanisme de sécurité : le ***learning rate*** (noté η): au lieu d'ajouter la totalité de la correction prédite par le Weak Learner, on n'en ajoute qu'une fraction (par exemple $\eta = 0.1$). Cela force l'algorithme à faire de nombreux "petits pas" prudents vers la solution, plutôt que quelques grands sauts risqués qui pourraient déstabiliser le modèle global.

Ce que les *Weak Learners* essaient de prédire et de corriger à chaque étape correspond en réalité au **gradient de la fonction de coût**(D'où le terme Gradient Boosting)

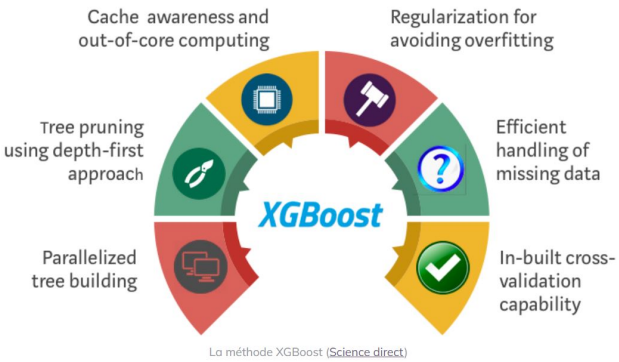


XGBoost est une variante des méthodes de Gradient Boosting qui utilise des arbres de décisions dit Gradient Boosting Tree comme prédicteur d'erreur. Il commence avec un prédicteur simple qui prédit un nombre arbitraire en générale 0.5 quelque soit l'entrée. Inutile de vous dire que ce prédicteur commet beaucoup d'erreurs.

Le Gradient Boosting est ensuite appliquée jusqu'à ce que l'erreur soit ramenée à un minimum.



XGBoost est une combinaison parfaite de techniques d'optimisation logicielle et matérielle permettant d'obtenir des résultats supérieurs en utilisant moins de ressources informatiques et en un minimum de temps



La méthode XGBoost (Science direct)

3 MÉTHODES D'APPRENTISSAGE NON SUPERVISÉS

L'**apprentissage non supervisé**(en anglais *Unsupervised Learning*) est une technique de Machine Learning qui fait appel à des algorithmes pour analyser des ensembles de données non étiquetées. Ceux-ci vont pouvoir révéler des motifs intrinsèques et effectuer des regroupements de données en fonction de leurs similitudes. Contrairement à l'apprentissage supervisé, il n'existe pas de fonction cible explicite à apprendre, mais plutôt des **patterns**, **clusters** ou représentations à découvrir

3.1 K-means clustering

Le **K-Means clustering** est un des algorithmes d'apprentissage non supervisé les plus populaires.Vous devez lui fournir un dataset non étiqueté et le nombre de clusters à obtenir. L'algorithme va démarrer par estimer les K **centroïdes**, en les générant de manière aléatoire ou en les choisissant directement dans le jeu de donnée. Cet algorithme va alors effectuer des itérations pour converger vers un résultat.On voit donc qu'il faut au préalable connaitre le nombre de clusters optimal.

Pour déterminer le nombre de cluster optimal,deux méthodes sont les plus utilisées:

1. La méthode **du COUDE**
2. La méthode **de la silhouette**

La méthode du COUDE

Cette méthode permet de trouver le nombre optimal de clusters K pour lequel les centroïdes ne bougent presque plus. Les étapes sont les suivantes:

- choisir la plage de k: $k \in \{1, 2, ..., K_{max}\}$ où K_{max} est souvent entre 8 et 20
- Pour chaque k, calculer K-means, et l'inertie: Pour calculer les K-means il faut:
  - fixeer K
  - initialiser les centroïdes(centres): cella se fait soit au hazard, soit à l'aide de K-means++ qui est la méthode la plus utilisée. On obtient ainsi k centres $\mu_1, \mu_2, ..., \mu_K$
  - affectation des points: Pour chaque x_i, on calcule la distance entre x_i et le centre μ_k. La distance la plus utilisée est la distance euclidienne:

$$d(x_i, \mu_k) = \sqrt{\sum_{j=1}^{p} (x_{i,j} - \mu_{k,j})^2}$$

Le point est affecté au cluster pour lequel cette distance est la plus petite

– Recalcul des centroïdes: on met à jour le centroïde μ_k:

$$\mu_k = \frac{1}{\mid C_k \mid} \sum_{i \in C_k} x_i$$

où C_k est le cluster k, $\mid C_k \mid$ est le nombre de points contenus dans le cluster(taille)

– On répète les deux étapes précédentes jusqu'à ce que les centres ne bougent presque plus, ou jusqu'à ce que le nombre maximal d'itération est atteint

– Pour le calcul de l'inertie, la formule est la suivante:

$$Inertie = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

- On trace le graphe $Inertie(K)$ qui est décroissant: l'inertie diminue rapidement au début, puis après un certain point, il diminue faiblement. Ce point en question est celui appelé le ***coude***. C'est à ce point qu'on obtient le nombre optimal de clusters

La méthode de la silhouette

Cette méthode quant à elle nous donne une valeur bien plus précise de notre K optimal, en maximisant une fonction dite la **fonction de silhouette**: Pour chaque point, elle détermine à quel point ce dernier est bien regroupé dans son cluster, et à quel point il est bien séparé des autres clusters. Sa formule est

$$s(i) = \frac{b(i) - a(i)}{max(a(i), b(i))}$$

où:

- $a(i)$ est la distance moyenne entre le point i et les autres points de son cluster
- $b(i)$ la plus petite distance moyenne entre i et les points des autres clusters

$s(i)$ est une fonction comprise dans $[-1, 1]$

fonctionnement de K_Means

Comme vu précédemment, le K-means repose sur l'idée qu'un cluster est représenté par un centroïde μ_k avec $k = 1, ..., K \quad Clusters$

L'objectif est de trouver une partition $C_1, C_2, ..., C_K$ de l'ensemble de nos données, et aussi de trouver les centroîdes μ_k de manière à minimiser la valeur

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k}$$

qui a été décrit plus haut comme étant l'**inertie** Après l'application de la méthode du coude, on se rend compte qu'on a déjà non seulement les centroïdes μ_k, mais aussi tous les points de chaque clusters (qui ont minimisés l'inertie)

Vu que à chaque itération la fonction d'inertie décroît en restant positive, on peut donc affirmer que la méthode de **K-Means clustering** converge

3.2 Réduction de Dimensionnalité: Analyse en Composantes Principales (ACP)

L'ACP, introduite par **Pearson** (1901) et **Hotelling** (1933),vise à réduire le nombre de variables ou de caractéristiques, tout en préservant autant d'informations que possible(variance).

Ici, on stocke notre jeu de données dans une matrice $n \times p$. Les étapes de l'ACP sont:

- On centre et on réduit nos données pour éviter à avoir À faire face à des problèmes d'échelle

$$X_s = \frac{X - \bar{X}}{\sigma}$$

- puis on calcule la matrice de covariance Σ selon la formule:

$$\Sigma = \frac{1}{n-1} X_s^T X_s$$

  On obtient donc une nouvelle matrice $p \times p$

- On effectue une décomposition en valeurs propres: c'est à dire qu'on cherche les valeurs propres λ_k et les vecteurs propres υ_k de la matrice Σ tels que:

$$\Sigma \upsilon_k = \lambda_k \upsilon_k$$

- On trie les valeurs propres par ordre croissants($\lambda_1 > \lambda_2 > ... > \lambda_p$) et à partir de ce classement on détermine les **axes principaux** $PC1$ et $PC2$
  - $PC1$ est l'axe dirigée par le vecteur unitaire υ_1 qui maximise la variance $var(X\upsilon_1) = \lambda_1$(la plus grande valeur propre) sous la contrainte $\|\upsilon_1\| = 1$
  - $PC2$ quant à lui va être dirigé par le vecteur υ_2, direction qui maximise la variance $var(X\upsilon_2)$ avec $\upsilon_1^T \upsilon_2 = 0$(orthogonalité)

- On calcule les nouvelle coordonnées (**scores**) de chaque individu grâce à la formule

$$Z = XV$$

  où V est la matrice des vecteurs propres

Ainsi, en choisissant les deux plus grandes valeurs propres, on se retrouve dans le plan qui explique le plus la variance: c'est le **plan factoriel**. La variance expliquée par chaque axe est de

$$Var_{expliquee_k} = \frac{\lambda_k}{\sum_{j=1}^{p} \lambda_j}$$

3.3 DBSCAN : Clustering Basé sur la Densité

Développé par **Ester et al.** en 1996, DBSCAN identifie les clusters comme des régions de haute densité.

DBSCAN repose sur trois notions essentielles:

- **voisinage** ϵ:Pour un point x son voisinage est :

$$N\epsilon(x) = y : d(x, y) \leq \epsilon$$

- **Point cœur (core point)**: Un point est core s'il y a assez de points autour de lui : $\mid N_\epsilon(x) \mid \geq MinPts$

- **Point frontière**: point qui n'est pas core, mais il est dans le voisinage d'un point cœur.

  Les points qui ne sont ni core ni frontières sont des **point bruit**.

3.3.1 Fonctionnement de DBSCAN

L'algorithme de DBSCAN est le suivant:

- Choisir un point non visité x
- Trouver ses voisins en calculant N_ϵ
- Vérifier si c'est un core point: si c'est le cas, c'est la création d'un nouveau cluster de centre ce point. Sinon, marquer temporairement ce point comme bruit
- Pour chaque point du voisinage, s'il est core, étendre en ajoutant ses voisins. Sinon, l'ajouter simplement

Cette méthode est très simple, et l'on n'a pas besoin de calculer un k optimal. En plus, elle repère automatiquement les points bruits. Toutefois, cette méthode est très sensible au choix de ϵ et a de mauvaises performances lorsque la taille de la base de données est très grande

4 MACHINE LEARNING SUR NOTRE BASE DE DONNÉES

Nous avons cherché une base de données auquel nous avons appliqué des méthodes supervisées comme non supervisées pour déduire des résultats. Notre base de donnée provient du site **Kaggle**(), et concerne Les requêtes d'assurance par les clients d'une compagnie d'assurance automobile.

4.0.1 DESCRIPTION ET OBSERVATION DE LA BASE DE DONNÉES

Les différentes variables de notre base de données sont: Ce qui nous donne un total de 41 colones pour

VARIABLE	DESCRIPTION	TYPE
policy_id	identifiant unique pour la police d'assurance	**objet**
subscription_length	La durée de validation de la police d'assurance	**float**
vehicle_age	L'âge du vehicule assuré, qui peut affecter la probabilité d'une demande d'assurance	**int**
customer_age	L'âge du client	**int**
Model	le modèle du vehicule qui peut impacter la frequence des demandes d'assurance due aux caractéristiques générales du modèle	**objet**
fuel_type	type d'essence que le vehicule utilise	**objet**
max_torque,max_power	performances du vehicule qui peuvent influencer ses conditions mécaniques	**objet**
engine_type	type d'engin	**objet**
displacement, cylinder...	spécification sur les dimensions du vehicule	**float**
region_code	la localisation géographique du client	**objet**
region_density	le nombre d'habitants dans la region du client	**int**
airbags	le nombre de airbags dans le vehicule	**int**
is_esc(Electronic stability Control),is_adjustable_streering,is_tpms(Tire Pressure Monitoring System)	varaiables qui spécifient les caractéristiques de sécurité du vehicule	**bool**
is_parking_sensors, is_parking_camera	caractéristiques d'aide pour le stationnement	**bool**
rear_brakes_type	type de freins, caractérisant la capacité d'arret du vehicule	**objet**
autres indicateurs binaires(yes/no)	pour spécifier les capacités du vehicule ainsi que ses mésures de sécurité	**bool**
claim_statut	indique si le client a déjà fait une demande d'ssistance(1) ou pas(0)	**bool**

58592 lignes, parmi lesquelles on a 13 variables numériques pour 28 variables catégorielles.

Après analyse de la base de données, nous constatons que nous n'avons aucune valeur manquante, et aucun doublons. Néanmoins, Nous constatons que les variables 'max_torque' et 'max_power', de manière réaliste, devraient être des variables numériques. Aussi, la colonne 'policy_id' ici nous est inutile. Tout

cela nous ramène à 15 variables nuériques pour 25 variables non numériques.

Nous avons également cherché à savoir si notre base de données présente des valeurs aberrantes et on a obtenu les résultats suivant:

```
nombre d'individus aux valeurs aberrantes par variable:

subscription_length:0<------>0.00%
vehicle_age:269<------>0.46%
customer_age:282<------>0.48%
region_density:3647<------>6.22%
max_torque:0<------>0.00%
max_power:0<------>0.00%
airbags:0<------>0.00%
displacement:0<------>0.00%
cylinder:0<------>0.00%
turning_radius:0<------>0.00%
length:0<------>0.00%
width:0<------>0.00%
gross_weight:0<------>0.00%
ncap_rating:0<------>0.00%
claim_status:3748<------>6.40%
```

Certaines variables présentent des valeurs aberrantes. Néamoins, ceux ci sont justifiées:

- **ages des véhicules**: min: 0.0 ——– max: 20.0: L'âge des véhicules sont entre 0(nouvelle voiture) et 20 ans: ce qui est tout à fait légitime. donc les valeurs aberrantes ne sont pas des erreurs. On les gardes
- **ages des clients**: min: 35 ——– max: 75 De même,les âges des clients vont de 35 ans à 75 ans: ceci est parfaitement légitime
- **populations**: min: 290——– max: 73430: Pour les populations, on n'a toujours pas d'erreur car , le nombre d'habitants ici vont de 290(petites zones) à 73430(grandes zones). Ce sont des réalités rencontrées donc rien est à changer sur ce côté.

Notre base de données est désormais propre. Passons maintenant à l'analyse exploratoire des données.

4.0.2 Analyse exploratoire des données

Nous allons effectuer une analyse univariée, puis une analyse bivariée, et en tirer des conclusions.

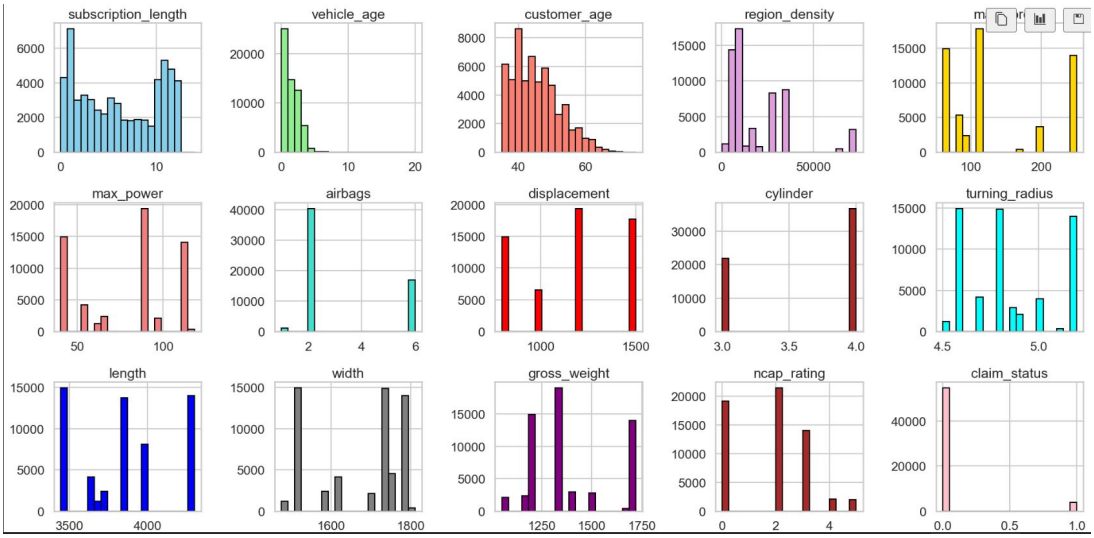
-ANALYSE UNIVARIÉE

Elle nous permet de connaitre et d'analyser les caractéristiques de chaque variable de manière individuelle.
Pour les variables numériques, les caractéristiques qu'on obtient sont:

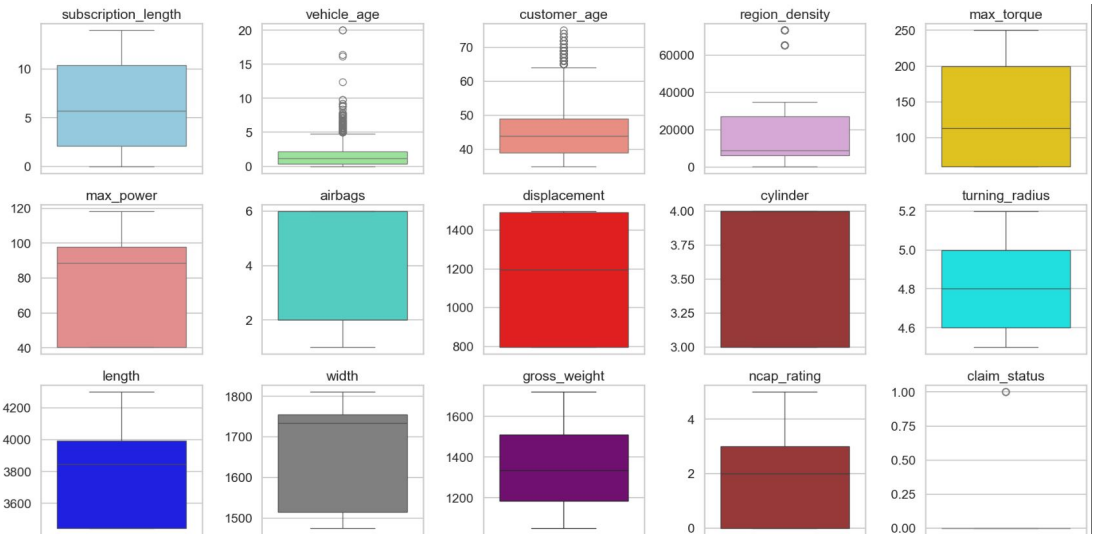
	subscription_length	vehicle_age	customer_age	region_density	max_torque	max_power	airbags	displacement	cylinder	turning_radius
count	58592.000000	58592.000000	58592.000000	58592.000000	58592.000000	58592.000000	58592.000000	58592.000000	58592.000000	58592.000000
mean	6.111688	1.388473	44.823935	18826.858667	134.450937	78.976765	3.137066	1162.355851	3.626963	4.852893
std	4.142790	1.134413	6.935604	17660.174792	73.146794	27.699259	1.832641	266.304786	0.483616	0.228061
min	0.000000	0.000000	35.000000	290.000000	60.000000	40.360000	1.000000	796.000000	3.000000	4.500000
25%	2.100000	0.400000	39.000000	6112.000000	60.000000	40.360000	2.000000	796.000000	3.000000	4.600000
50%	5.700000	1.200000	44.000000	8794.000000	113.000000	88.500000	2.000000	1197.000000	4.000000	4.800000
75%	10.400000	2.200000	49.000000	27003.000000	200.000000	97.890000	6.000000	1493.000000	4.000000	5.000000
max	14.000000	20.000000	75.000000	73430.000000	250.000000	118.360000	6.000000	1498.000000	4.000000	5.200000

length	width	gross_weight	ncap_rating	claim_status
58592.000000	58592.000000	58592.000000	58592.000000	58592.000000
3850.476891	1672.233667	1385.276813	1.759950	0.063968
311.457119	112.089135	212.423085	1.389576	0.244698
3445.000000	1475.000000	1051.000000	0.000000	0.000000
3445.000000	1515.000000	1185.000000	0.000000	0.000000
3845.000000	1735.000000	1335.000000	2.000000	0.000000
3995.000000	1755.000000	1510.000000	3.000000	0.000000
4300.000000	1811.000000	1720.000000	5.000000	1.000000

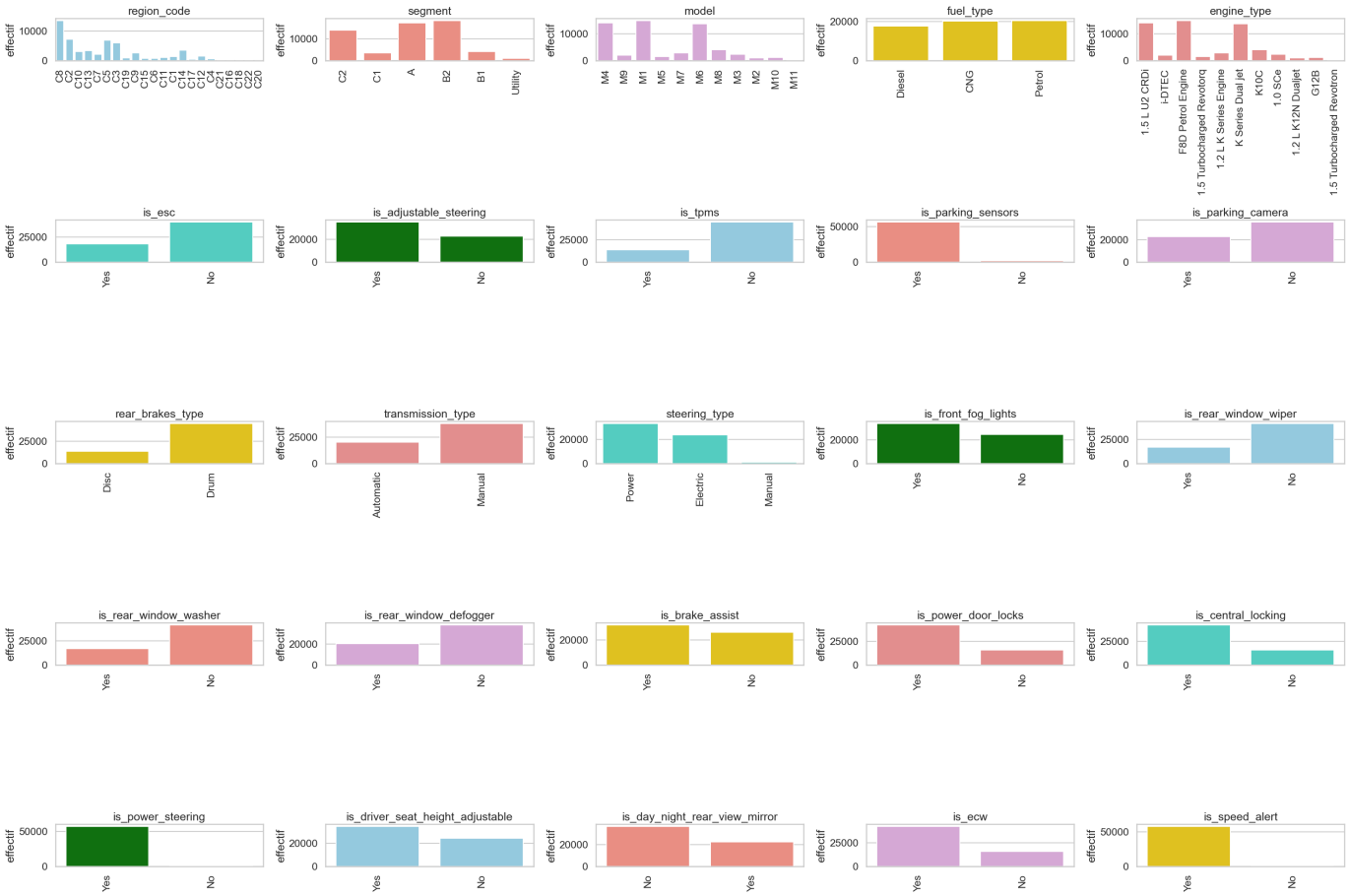
Nous passons maintenant à la visualisation de ces variables numériques grâce à des histogrames:



Nous tracons également les différents boxplots et on obtient



En ce qui concerne les variables qualitatives, nous allons tracer les diagrammes en batons pour pouvoir non seulement les visualiser, mais aussi de determiner des paramètres tel que le mode de chaque variable

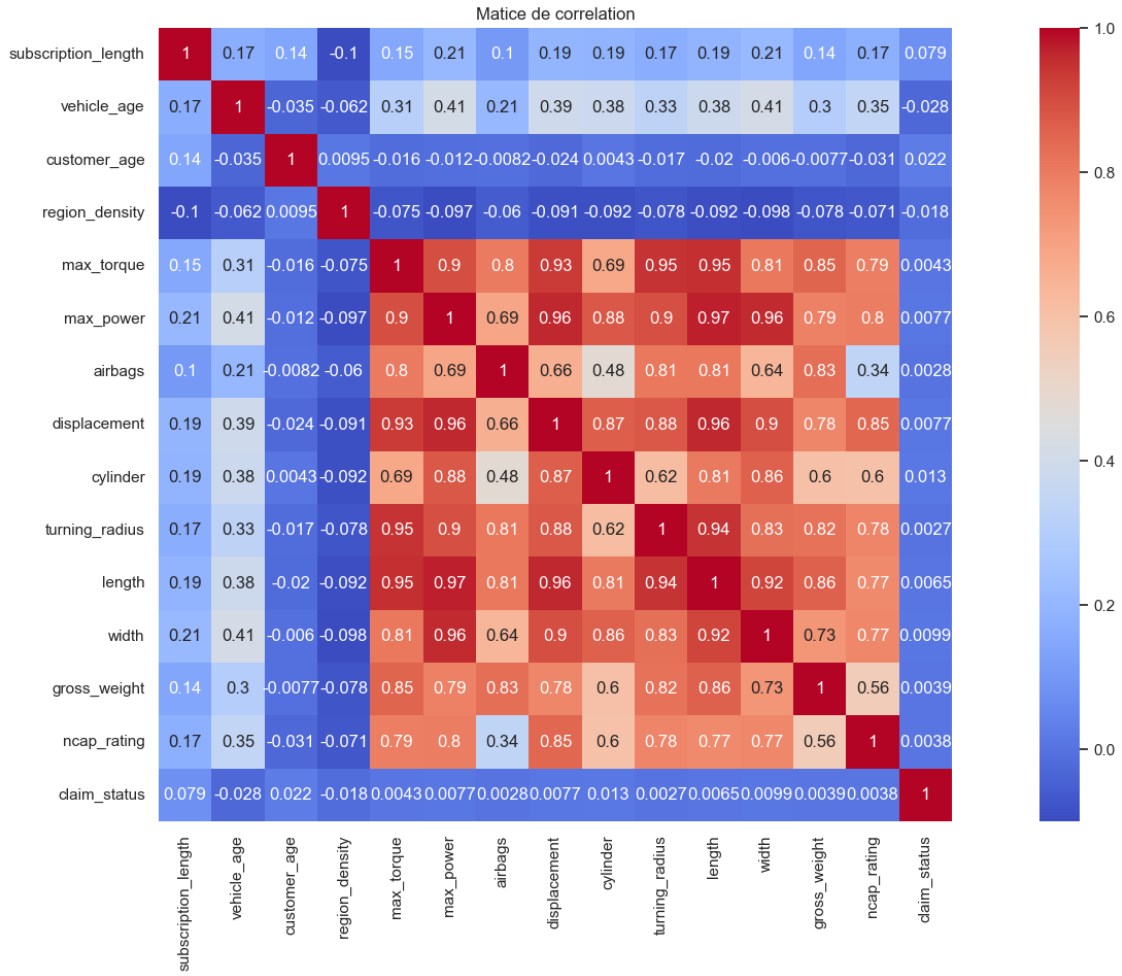


-ANALYSE BIVARIÉE

Dans cette partie, nous allons regarder les différentes relations entre nos variables.

Numérique $\longleftrightarrow$ Numérique

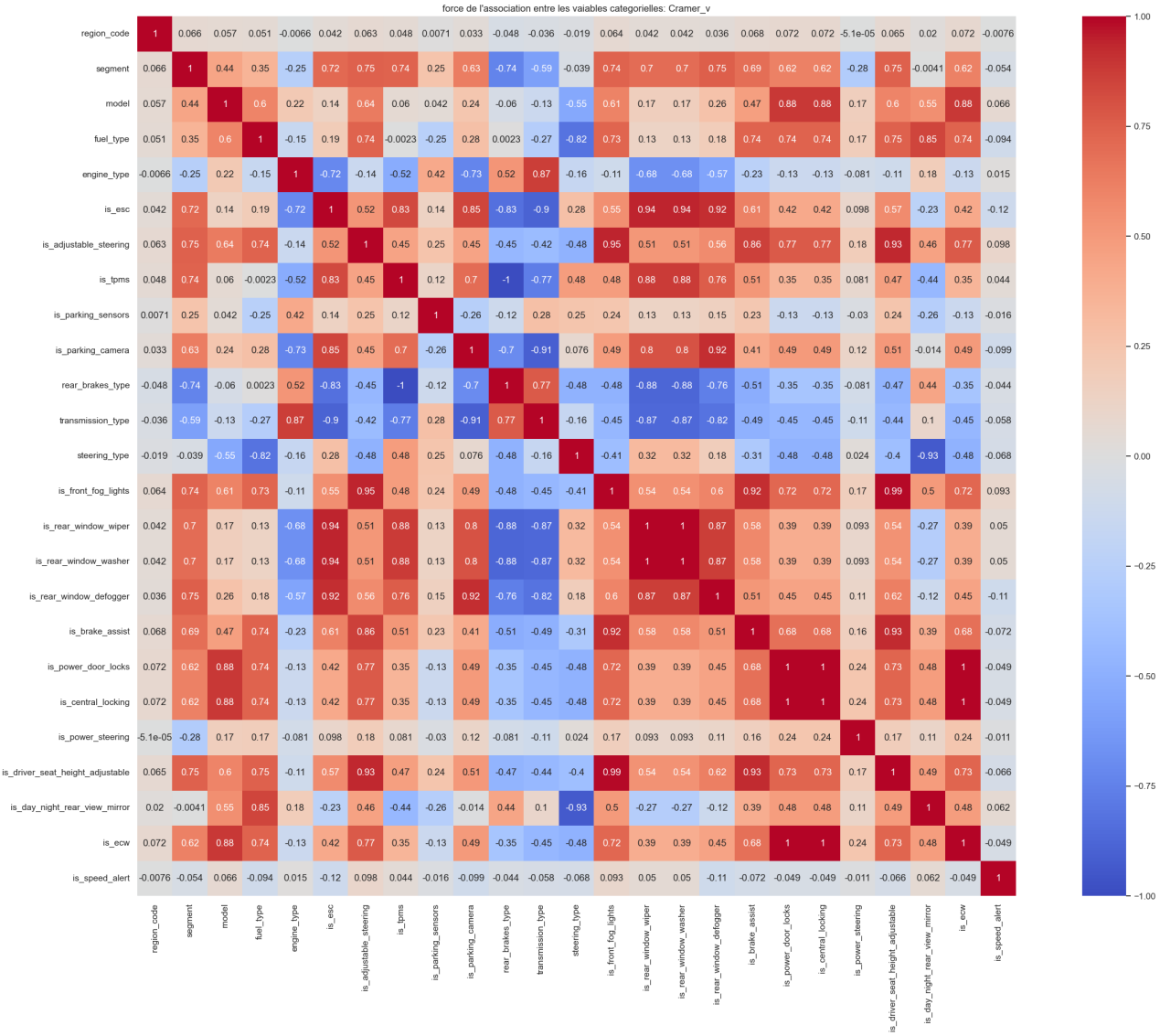
Pour les variables numériques, nous allons déterminer la matrice de corrélation. On obtient la matrice suivante:



- On constate qu'on n'a pas vraiment de forte corrélation négative, mais beaucoup de forte corrélation positive entre nos variables numérique, particulièrement entre les variables :**'airbags', 'displacement', 'cylinder', 'turning_radius', 'length', 'width', 'gross_weight', 'ncap_rating','max_power','max_torque'**
- On se rends aussi compte que notre variable cible est très faiblement corrélée avec les autres

catégorielle $\longleftrightarrow$ catégorielle

Nous allons calculer les coefficient de v de cramer pour déterminer la force de relation entre les variables catégorielle, dans une seule et grande matrice



Plus la mesure est proche de 1, plus l'association entre les variables est forte. Plus il est proche de zero, plus l'association est faible

Regardons maintenant comment se comporte notre variable cible avec les autres variables:

```
claim_status                        1.000000
subscription_length                 0.078739
customer_age                        0.022234
is_adjustable_steering              0.013917
cylinder                            0.013434
is_front_fog_lights                 0.011825
is_brake_assist                     0.010893
is_driver_seat_height_adjustable    0.010686
fuel_type                           0.009612
region_code                         0.009425
displacement                        0.008511
is_parking_sensors                  0.008419
width                               0.008083
is_day_night_rear_view_mirror       0.007989
max_torque                          0.007407
is_speed_alert                      0.007307
length                              0.007052
is_power_door_locks                 0.006637
is_central_locking                  0.006637
is_ecw                              0.006637
model                               0.006548
max_power                           0.006396
segment                             0.006366
ncap_rating                         0.003211
airbags                             0.003085
```

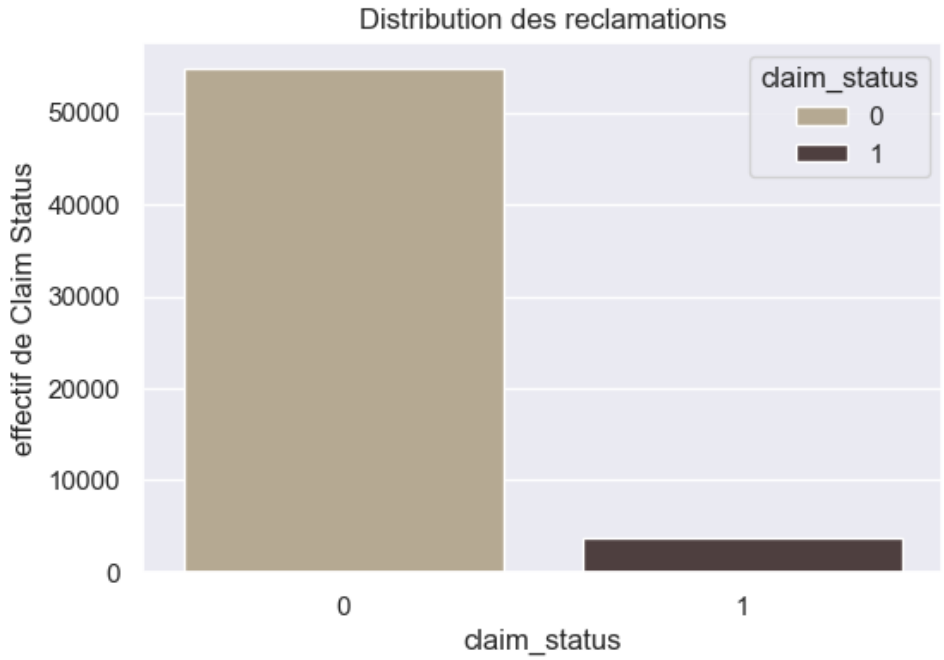
La question qu'on pourrais se poser c'est pourquoi notre variable cible **claim_status** est si faiblement corrélée avec les autres variables.
En effet, nous faisons façe à un problème appelé le **déséquilibre des classes**

Le **déséquilibre des classes** est un problème omniprésent en ingénierie financière et actuarielle: lorsque la classe minoritaire représente un enjeu critique pour l'entreprise (par exemple, un défaut de prêt coûte beaucoup plus cher qu'un prêt accordé), un modèle entraîné sur des données déséquilibrées aura tendance à simplement prédire la classe majoritaire pour obtenir une précision globale élevée, mais il sera catastrophique pour la classe minoritaire.

Pour notre cas les proportions entre les classes sont:

```
Comptes bruts :
claim_status
0    54844
1     3748
Name: count, dtype: int64

Pourcentages pour les reclamations :
claim_status
0    93.603222
1     6.396778
```



On voit clairement que nos données de la classe '**claim_statut**'=1 sont fortement minoritaires par rapport à l'autre classe. Les modèles classiques ne peuvent donc pas nous aider ici pour faire nos classification.

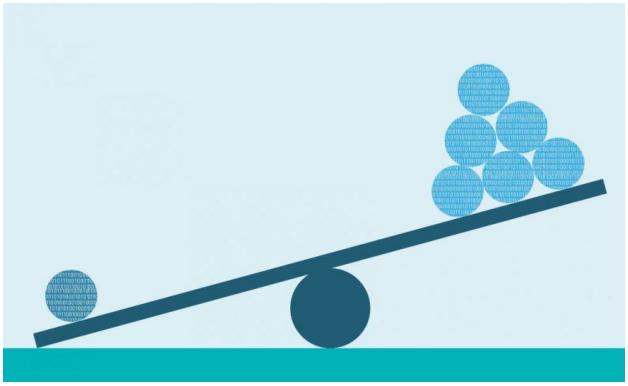
4.0.3 Préparation de nos données pour le machine learning

Pour faire du machine learning, les données doivent tout d'abord respecter certain standards.Nous allons donc préparer notre base de données, en suivant les étapes ci-apres:

- **Étape 1**: Séparation de la variable cible: On sépare notre variable cible du reste de la base de données
- **Étape 2**: Identification des variables par types: Nous allons séparer notre base de données par type de variables: numérique, nominale et binaire(yes/no)
- **Étape 3**: Encodage des variables binaires: Nous allons changer le format 'yes/no' en un format '1/0' où 1='yes' et 0='No'
- **Étape 4**: Encodage des variables nominales: nous allons aussi encoder nos variables nominales en créant de nouvelle colonnes binaire pour chaque valeur. Ici, on utilise le paramètre **drop_first=True** pour supprimer les valeurs de moindre importance pour ne pas élargir la base de données inutilement
- **Étape 5:** Normaliser les variables numériques: Pour résoudre le problème d'échelles entre les variables

- **Étape 6**: Séparation des données de test et d'entrainement. Ici, on utilise **stratify=y** pour garantir que la répartition des classes dans les données d'entrainement et de test reste la même

4.0.4 Traitement du déséquilibre (suréchantillonage)



Un modèle classique (comme la Régression Logistique ou un Arbre de Décision simple) entraîné sur ces données sera biaisé en faveur de la classe majoritaire. Il apprendra très bien à reconnaître la classe majoritaire, mais son pouvoir de prédiction sur les cas minoritaires importants sera faible.

Pour palier à ce problème, les financiers et actuaires adoptent très souvent des méthodes de **rééchantillonnage**: son objectif est de créer un jeu de données d'entraînement équilibré pour que l'algorithme accorde une importance plus ou moins égale aux deux classes

On distingue deux types de rééchantillonnage:

- Méthodes de **Sous-Échantillonnage (Undersampling)** : consiste à réduire le nombre d'observations de la classe majoritaire afin qu'il corresponde au nombre d'observations de la classe minoritaire.
- Méthodes de **Sur-Échantillonnage (Oversampling)** meilleur que le Sous-Échantillonnage, il consiste à augmenter le nombre d'observations de la classe minoritaire jusqu'à ce que les classes soient équilibrées.

Chaque méthode présente ses avantages comme ses inconvénients comme la **perte d'information** pour le Sous-Échantillonnage ou encore le **Risque de surapprentissage** (Overfitting) pour le Sur-Échantillonnage

Dans notre cas, nous allons travailler dans un premier temps avec la base de données initiale, puis nous allons y appliquer un Sur-Échantillonnage simple, et enfin, nous allons appliquer une variante de Sur-Échantillonnage, pour pouvoir bien visualiser l'impact de tout cela sur nos performances.

Après suréchantillonage, on caonstate que notre base de données en maintenant équilibrée:

```
claim_status
0    43875
1    43875
Name: count, dtype: int64
```

Nous avons parlé de la variante de suréchantillonage que nous allons appliquer. De quoi s'agit il exactement?
Cette méthode est encore appelée **Synthetic Minority Oversampling TEchnique** ou tout simplement **SMOTE**.Cette méthode présente de bonnes performances lorsqu'on effectue une classification. Son principe est le suivant:

1. Pour chaque observation x_i de la classe minoritaire: L'algorithme identifie ses k plus proches voisins (par exemple, $k = 5$ par défaut) parmi les autres observations minoritaires.
2. Il sélectionne un voisin aléatoire, x_{zn}, parmi ces k voisins
3. Il génère un nouveau point synthétique x_{new} sur le segment de droite reliant x_i et x_{zn}. La position du nouveau point est déterminée par une interpolation aléatoire.

$$x_{new} = x_i + (\text{différence entre } x_{zn} \text{ et } x_i) \times \delta$$

Où δ est un nombre aléatoire entre 0 et 1.

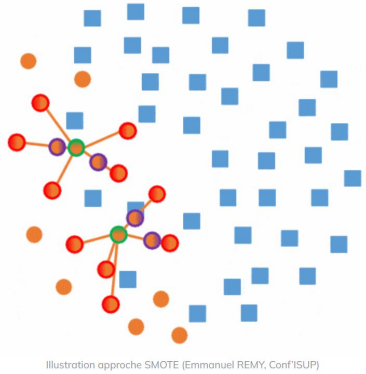


Illustration approche SMOTE (Emmanuel REMY, Conf'ISUP)

appliquer SMOTE sur tout le dataset créerait du **data leakage** (les données synthétiques influenceraient le test et on aurais de fausse très bonnes performances), donc on l'applique uniquement sur nos données d'entrainement

4.1 MODÈLE SUPERVISÉ: CLASSIFICATION

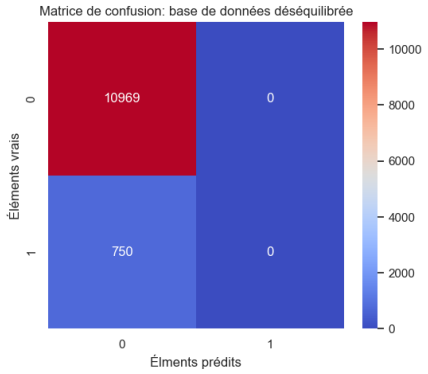
Passons maintenant à notre tout premier type de modèle: les **modèles supervisés**. Pour ce type de modèle, vu que nous avons à faire à une classification, on va faire une **regression logistique**.

4.1.1 Classification avec une regression logistique

Comme expliqué plus haut, la **regression logistique** permet de modéliser la relation entre une variable binaire et des variables indépendantes. Nous avons écris sa fonction python suivant les étapes mathématiques décrites plus haut, et nous allons l'appliquer sur nos trois versions de notre bases de données(base de données initiale, simplement suréchantillonée, et suréchantillonée avec SMOTE)

CAS DE LA BASE DE DONNÉES INITIALE

Dans ce cas ci, on obtient les métriques suivantes:




```
AUC: 0.6261042331418846
Rapport complet :
              precision    recall  f1-score   support

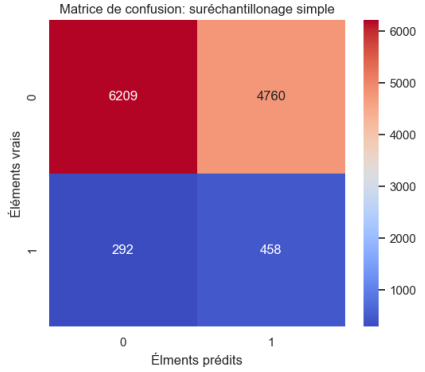
           0       0.94      1.00      0.97     10969
           1       0.00      0.00      0.00       750

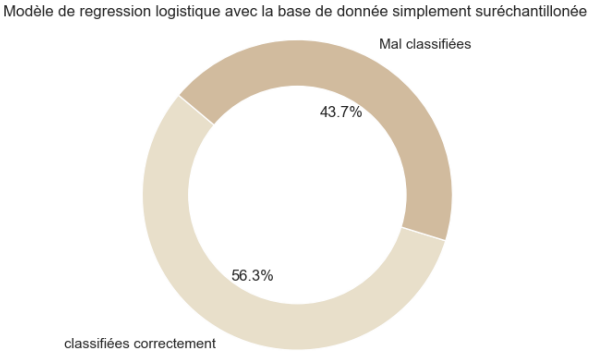
    accuracy                           0.94     11719
   macro avg       0.47      0.50      0.48     11719
weighted avg       0.88      0.94      0.91     11719
```

Comme nous l'avions espéré, notre modèle est biasée vers la classe majoritaire. Pour lui, tous les individus sont dans la classe 0(n'ont pas encore fait de requête auprès de la compagnie d'assurance).Ceci est la raison pour laquelle nous devons chercher d'autres alternatives.

CAS DE LA BASE DE DONNÉES SIMPLEMENT SURÉCHANTILLONÉE

Les résultats obtenus ici sont:



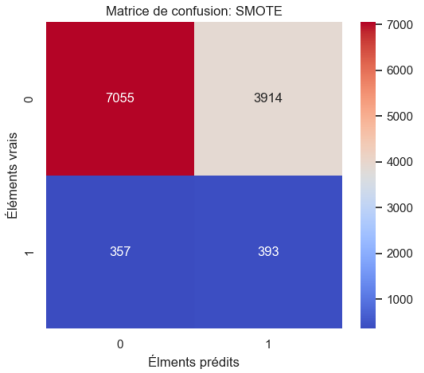


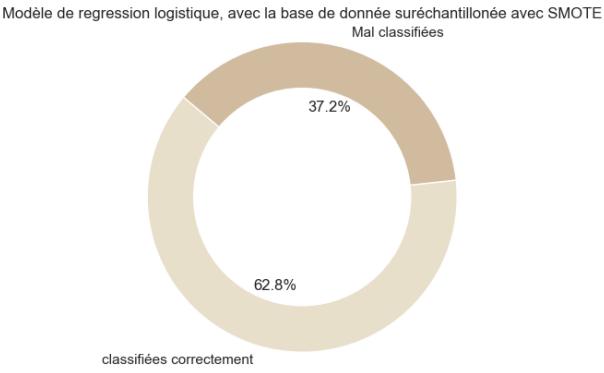

```
AUC: 0.6261042331418846
Rapport complet :
              precision    recall  f1-score   support

           0       0.96      0.57      0.71     10969
           1       0.09      0.61      0.15       750

    accuracy                           0.57     11719
   macro avg       0.52      0.59      0.43     11719
weighted avg       0.90      0.57      0.68     11719
```

CAS DE LA BASE DE DONNÉES SURÉCHANTILLONÉE AVEC SMOTE






```
AUC: 0.6126718327407541
Rapport complet :
              precision    recall  f1-score   support

           0       0.95      0.64      0.77     10969
           1       0.09      0.52      0.16       750

    accuracy                           0.64     11719
   macro avg       0.52      0.58      0.46     11719
weighted avg       0.90      0.64      0.73     11719
```

INTERPRÉTATION ET COMPARAISON DES MÉTRIQUES

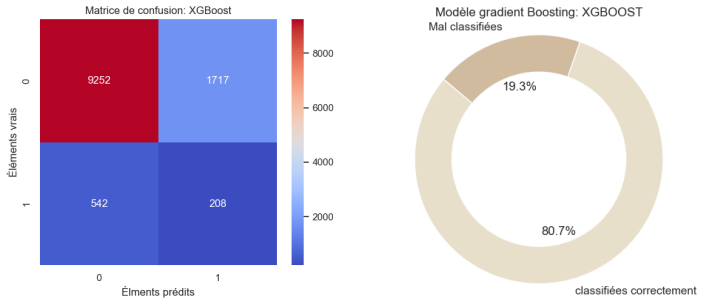
Les performances et interprétation sont récapitulés dans le tableau ci dessous

MÉTRIQUE	VALEUR1 (suréchantillonage simple)	VALEUR2(SMOTE)	VALEUR3(déséquilibrée)	INTERPRÉTATIONS
Accuracy (Exactitude): Proportion de prédictions correctes	0.57	0.64	0.94	Ces valeurs ne sont pas très bonnes dans les deux premiers cas, mais sont un peu meilleures avec SMOTE.Dans le cas du déséquilibre, la valeur est biasée comme dit plus haut
Precision (Précision): juge combien sont réellement positifs parmi ceux que le modèle prédit positifs	0.96(classe 0) et 0.09(classe 1)	0.95(classe 0) et 0.09(classe 1)	0.94(classe 0) et 0.00(classe 1)	Nos modèles font de bonnes précisions pour la classe majoritaire mais pas pour la classe minoritaire(peu de vrai positifs parmis les positifs)
Recall (Sensibilité): combien ont été détectés parmi les vrais positifs réels	0.57(classe 0) et 0.61(classe 1)	0.64(classe 0) et 0.52(classe 1)	1.00(classe 0) et 0.00(classe 1)	Environ la moitié des vrai positifs ont été détectés(performances pas très satifaisantes) pous les deux modèles avec suréchantillonage(bien que SMOTE soit un peu plus performant). Pour le cas déséquilibré, on a aucun faux negatifs
F1-Score: moyenne harmonique entre Precision et Recall.	0.71(classe 0) et 0.15(classe 1)	0.77(classe 0) et 0.16(classe 1)	0.97(classe 0) et 0.00(classe 1)	Pous la classe majoritaire, on a un bon compromis, ce qui n'est pas le cas pour la classe minoritaire. Le deuxième modèle est un peu plus performant que le premier. Dans le cas où la bd est déséquilibrée, cette metrique nous indique un modèle presque parfait pour la classe 0(tous les individus du la classe 0 ont été bien prédits)
ROC-AUC Score: Mesure la capacité du modèle à séparer les classes	0.6261	0.6228	0.6261	Ces valeurs ne sont pas très bonnes dans les deux cas(< 0.7). Elles signalent une situation de **hazard**

Nous pouvons conclure, au vu des résultat, que ce modèle n'est pas adapté pour notre base de données. Nous devons trouver une autre solution

4.1.2 Classification avec un Gradient Boostiong(XGBoost)

Nous cherchons une méthode plus efficace que le modèle de régression logistique. Or nous avons dit plus haut que la méthode de Gradient Boostiong(XGBoost) a un algorithme très performant. Nous allons donc appliquer cette méthode, en utilisant le XGBoost de python, sur notre base de données. On obtient les métriques suivantes:




```
AUC: 0.6423065609140913
Rapport complet :
              precision    recall  f1-score   support

           0       0.94      0.84      0.89     10969
           1       0.11      0.28      0.16       750

    accuracy                           0.81     11719
   macro avg       0.53      0.56      0.52     11719
weighted avg       0.89      0.81      0.84     11719
```

On a les performances suivantes

MÉTRIQUE	VALEUR	INTERPRÉTATIONS
Accuracy (Exactitude): Proportion de prédictions correctes	0.82	On a une bonne précision
Precision (Précision): juge combien sont réellement positifs parmi ceux que le modèle prédit positifs	0.94(classe 0) et 0.11(classe 1)	Nos modèles font de bonnes précisions pour la classe majoritaire mais a toujours des difficultés avec la classe minoritaire(peu de vrai positifs parmis les positifs pour cette classe), bien que ces performances sont meileure par rapport aux cas précédents
Recall (Sensibilité): combien ont été détectés parmi les vrais positifs réels	0.86(classe 0) et 0.28(classe 1)	meilleures performances que précedemment
F1-Score: moyenne harmonique entre Precision et Recall.	0.90(classe 0) et 0.16(classe 1)	Pous la classe majoritaire, on a un très bon compromis, ce qui n'est pas le cas pour la classe minoritaire
ROC-AUC Score: Mesure la capacité du modèle à séparer les classes	0.6423	Ces valeurs ne sont pas très bonnes(< 0.7). Elles signalent une situation de **hazard**(certainement due à SMOTE)

4.1.3 classification d'un nouvel individu dans une des classes

Nous avons conçu une interface python qui permettra aux individus d'entrer leurs information et d'avoir la classe à laquelle ils appartiennent suivant le modèle XG-BOOST(le plus performant actuellement)

L'interface se présente comme suit:



à la sortie, l'individu connait la classe à laquelle il appartient

4.1.4 Conclusion Partielle

Dans cette partie du projet, plusieurs stratégies de modélisation ont été explorées afin d'évaluer l'impact du déséquilibre des classes et des techniques de rééquilibrage sur la performance des modèles de classification.

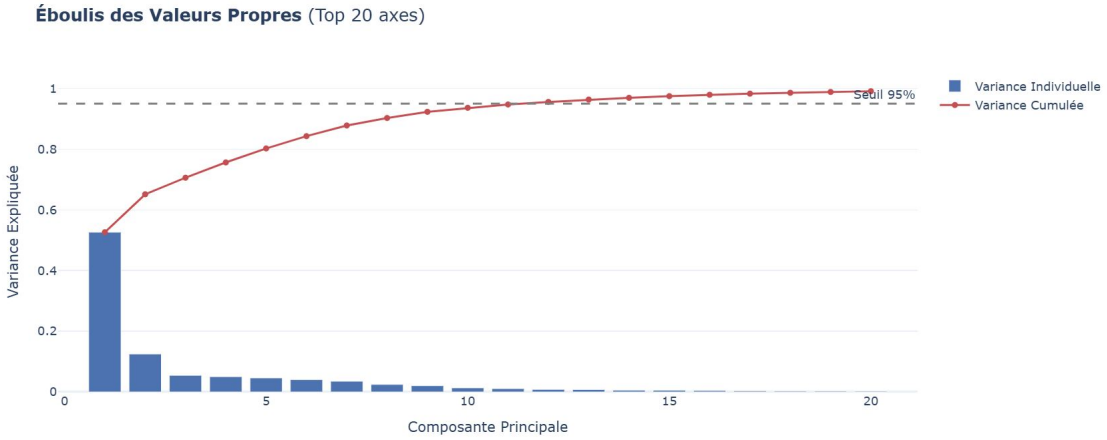
- L'application de la régression logistique directement sur la base de données déséquilibrée a rapidement mis en évidence les limites d'un modèle linéaire en présence d'un fort déséquilibre
- Un suréchantillonnage simple de la classe minoritaire a permis d'améliorer sensiblement la sensibilité du modèle (recall sur la classe minoritaire) au prix d'un léger risque de surapprentissage. Bien que rudimentaire, ce rééquilibrage a montré que la performance du modèle dépend fortement de la distribution des classes.
- L'utilisation de SMOTE, une technique de suréchantillonnage synthétique plus avancée a démontré une meilleure stabilité du modèle ainsi qu'un compromis plus équilibré entre précision et rappel. La régression logistique se comporte dans ce cas de manière plus robuste, ce qui confirme l'intérêt des techniques de rééquilibrage sophistiquées dans les contextes de déséquilibre sévère.
- Enfin, le modèle XGBoost a été testé sur la base rééquilibrée. Cet algorithme d'ensemble, fondé sur des arbres boostés, s'est distingué par sa capacité à capturer des relations complexes et des interactions non linéaires entre les variables. Il a fourni les meilleures performances globales, avec des métriques supérieures à celles obtenues par la régression logistique. XGBoost s'avère donc plus adapté dès que la structure des données dépasse le cadre linéaire

4.2 MODÈLE NON SUPERVISÉ: K-MEANS

Nous appliquons maintenat un modèle non supervisé À notre base de données: La méthode des **K-MEANS**. Elle a été décrite plus haute comme un modèle très populaire, simple à utilisé, et surtout performant.

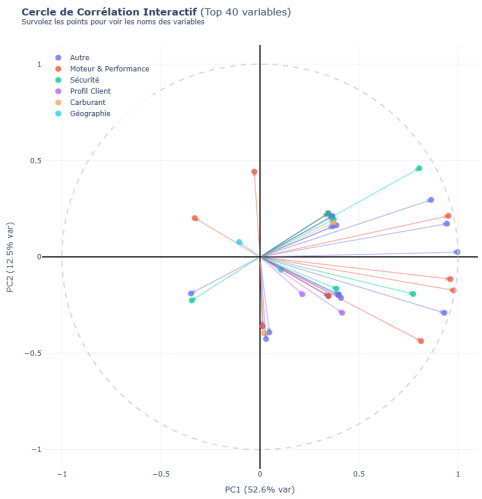
Avant toute chose nous allons passer par une étape préparatoire de la base de données. Ici, nous choisissons de réduire tout d'abord la dimension de notre base de données(bien sûr en gardant le plus possible d'information). Pour le faire, nous appliquons la méthode de réduction des dimensions par **analyse en composantes**

principales(ACP) qui a été décrite plus haut. En suivant les étapes mathématiques, on trace l'histogramme des valeurs propres avec leurs variances expliquée suivante: On visualise très bien les plus grandes valeurs propres. On constate que,



sur toutes les valeurs propres qu'on a, seules 11 suffisent à expliquer déjà jusqu'à 95% de l'information totale, et les deux premières suffisent à expliquer plus de 65% de la variance. Notre plan factoriel sera donc bien représentatif.

On trace aussi le **cercle de corrélation** qui est très important lorsqu'on effectue une ACP.On obtient la figure ci après:



L'analyse de notre figure nous permet de donner les interprétations suivantes:

- **Longueur du vecteur**: Les variables dont les vecteurs dont "longs"(proches du bord du cercle) sont les variables bien représentées par les axes(axe PC1 et PC2): On peut citer **cylinder, width, max_power, length, turning_radius, max_torque, gross_weight, airbags...**
- **Angle entre deux vecteurs**: ces angles définissent la corrélation entre les

variables en question.

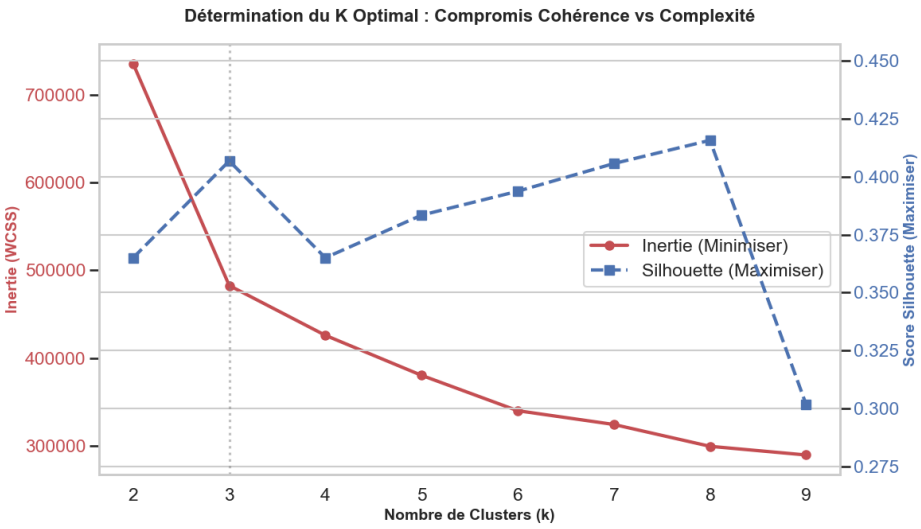
- angle $\approx 0 \longrightarrow$ les variables en question ont une forte corrélation positive: **is_tpms, is_esc, is_rear_window_washer , fuel_type_diesel**; **region_density,engine_type_F8D Petrol Engine ...**
- angle ≈ 180: $\longrightarrow$ les variables en question ont une forte corrélation négative: **lenght,engine_type_F8D Petrol_Engine**; **airbags, rear_brakes _type_Drum** ...
- angle $\approx 90°$ $\longrightarrow$Les variables en question sont indépendantes: **lenght, steering _type_Power**;
  engine_type_F8D Petrol Engine,airbags; **lenght, fuel_type_petrol**...

- **Projection sur les axes**: Définissent les contributions aux axes.
  - les variables qui contribuent fortement à l'axe PC1 sont: **lenght,displacement,...**
  - les variables qui contribuent fortement à l'axe PC2 sont: **is_day_night_rear _view_mirror, steering_type_Power,...**

Nous avons terminé avec l'analyse en composantes principales. nous allons donc continuer avec notre méthode principale qui est K_means

Recherche du nombre optimal de clusters (K)

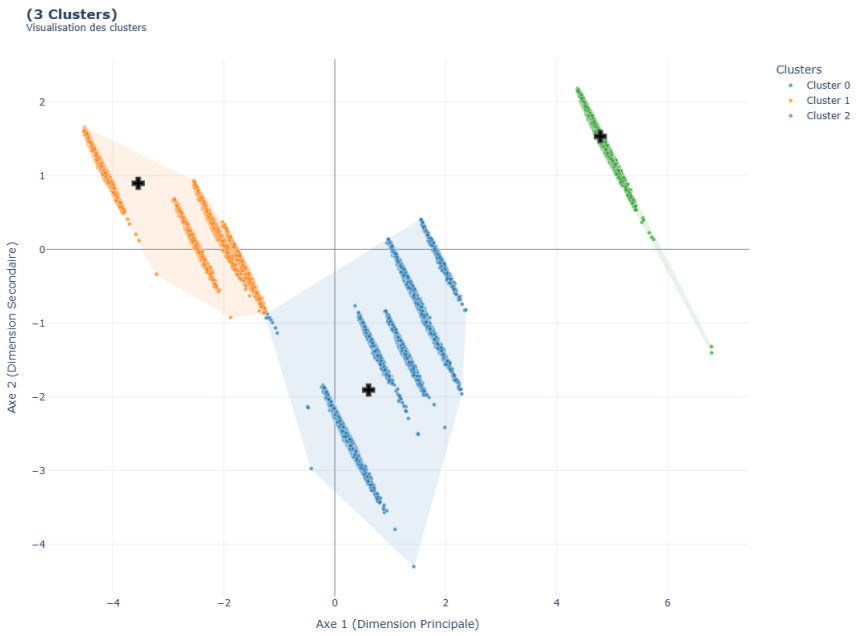
Le nombre optimal K de clusters est un facteur très important dans l'application de la méthode K-means. Pour le trouver, nous allons appliquer les deux méthodes mentionnées plus hauts: La méthode du COUDE et la méthode de la silhouette. Nous allons déterminer le résultat de façon graphique:



On voit que , les deux méthodes nous donnent un k optimal de $K_{optimal} = 3$. C'est cette valeur que nous allons utiliser pour visualiser nos clusters.

4.2.1 Visualisation des clusters

Maintenant, nous savons qu'il y a trois clusters. Nous allons donc diviser notre base de données en trois groupes en fonction des clusters, et afficher les différents clusters dans un graphe. On obtient la courbe suivante:



On voit clairement nos trois clusters, ainsi que leurs centroïdes. Ce qu'il nous reste à faire c'est de donner la particularité de chaque cluster.

Pour le faire, nous faisons une analyse exploratoire sur les trois sous bases de données (une sous base par cluster).On a les résultats suivants:

Base de données du Cluster 0(celui en bleue)

	subscription_length	vehicle_age	customer_age	region_density	max_torque	max_power	airbags	displacement	cylinder	turning_radius
count	21882.000000	21882.000000	21882.000000	21882.000000	21882.000000	21882.000000	21882.000000	21882.000000	21882.000000	21882.000000
mean	6.722955	1.700183	44.948359	17922.359611	128.691856	89.910182	2.537382	1247.930308	3.982954	4.836011
std	4.227493	1.110738	7.120460	16617.747841	33.045374	4.650261	1.364208	112.916386	0.129446	0.066627
min	0.000000	0.000000	35.000000	290.000000	82.100000	55.920000	1.000000	998.000000	3.000000	4.500000
25%	2.500000	0.800000	39.000000	7788.000000	113.000000	88.500000	2.000000	1197.000000	4.000000	4.800000
50%	6.900000	1.600000	44.000000	8794.000000	113.000000	88.500000	2.000000	1197.000000	4.000000	4.800000
75%	10.800000	2.400000	49.000000	27003.000000	113.000000	88.500000	2.000000	1197.000000	4.000000	4.850000
max	13.900000	20.000000	75.000000	73430.000000	200.000000	118.360000	6.000000	1498.000000	4.000000	5.100000

length	width	gross_weight	ncap_rating	claim_status	clusters
21882.000000	21882.000000	21882.000000	21882.000000	21882.000000	21882.000000
3899.353898	1735.121104	1334.293209	2.193264	0.067864	0.000000
71.155455	17.499541	109.853855	1.301279	0.251518	0.000000
3655.000000	1475.000000	1051.000000	0.000000	0.000000	0.000000
3845.000000	1735.000000	1335.000000	2.000000	0.000000	0.000000
3845.000000	1735.000000	1335.000000	2.000000	0.000000	0.000000
3990.000000	1735.000000	1335.000000	2.000000	0.000000	0.000000
3995.000000	1811.000000	1660.000000	5.000000	1.000000	0.000000

Base de données du Cluster 1(celui en orange)

	subscription_length	vehicle_age	customer_age	region_density	max_torque	max_power	airbags	displacement	cylinder	turning_radius
count	22692.000000	22692.000000	22692.000000	22692.000000	22692.000000	22692.000000	22692.000000	22692.000000	22692.000000	22692.000000
mean	5.091966	0.844491	44.726908	20825.987793	68.623925	47.137755	1.946765	875.580337	3.053235	4.654746
std	3.779593	0.912108	6.731759	19403.428571	12.199898	9.852161	0.224506	118.237697	0.224506	0.126687
min	0.000000	0.000000	35.000000	290.000000	60.000000	40.360000	1.000000	796.000000	3.000000	4.500000
25%	1.500000	0.200000	40.000000	5410.000000	60.000000	40.360000	2.000000	796.000000	3.000000	4.600000
50%	4.600000	0.600000	44.000000	13051.000000	60.000000	40.360000	2.000000	796.000000	3.000000	4.600000
75%	8.300000	1.200000	49.000000	34738.000000	82.100000	55.920000	2.000000	998.000000	3.000000	4.700000
max	14.000000	9.800000	74.000000	73430.000000	91.000000	67.060000	2.000000	1196.000000	4.000000	5.000000

length	width	gross_weight	ncap_rating	claim_status	clusters
22692.000000	22692.000000	22692.000000	22692.000000	22692.000000	22692.000000
3525.651331	1538.840781	1227.665036	0.576062	0.060021	1.000000
113.767113	44.387711	91.174476	0.905711	0.237531	0.000000
3445.000000	1475.000000	1155.000000	0.000000	0.000000	1.000000
3445.000000	1515.000000	1185.000000	0.000000	0.000000	1.000000
3445.000000	1515.000000	1185.000000	0.000000	0.000000	1.000000
3655.000000	1579.000000	1185.000000	2.000000	0.000000	1.000000
3731.000000	1620.000000	1510.000000	2.000000	1.000000	1.000000

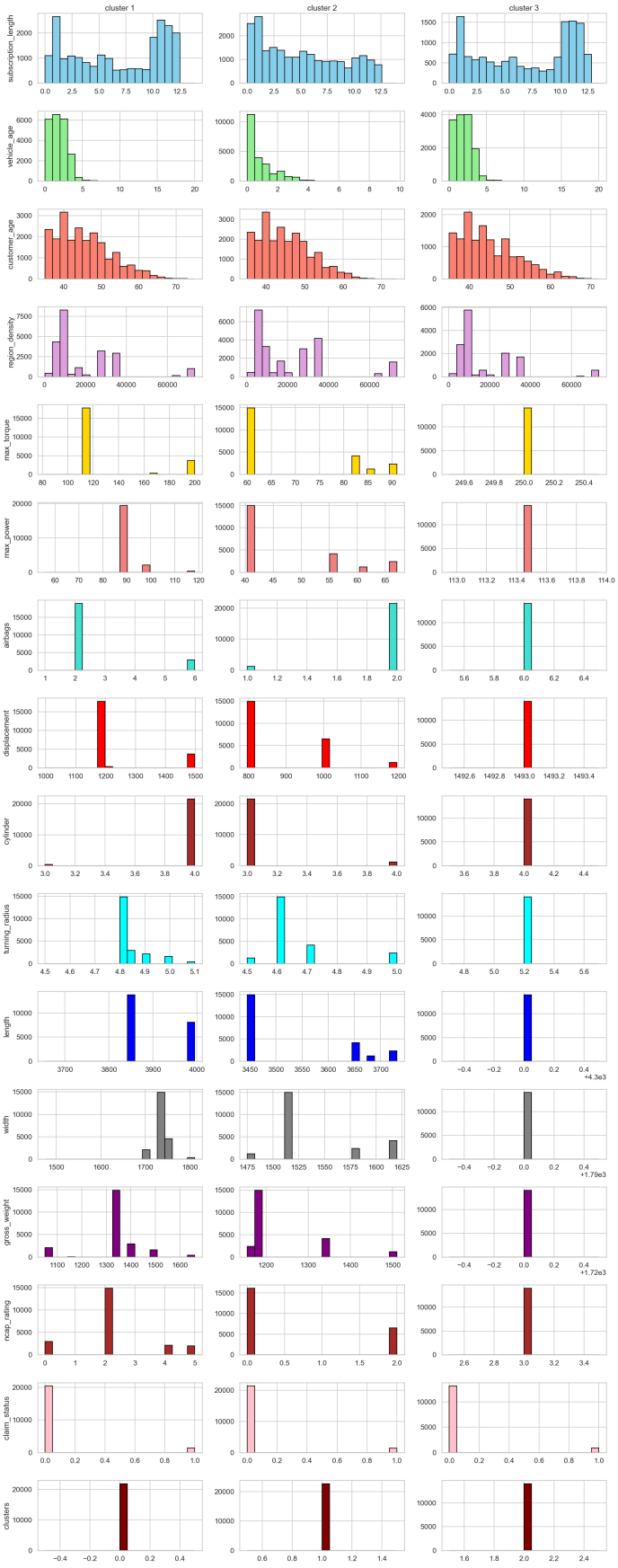
Base de données du Cluster 2(celui en vert)

	subscription_length	vehicle_age	customer_age	region_density	max_torque	max_power	airbags	displacement	cylinder	turning_radius
count	14018.000000	14018.000000	14018.000000	14018.000000	14018.000000	14018.000000	14018.000000	14018.000000	14018.000000	14018.000000
mean	6.808204	1.782480	44.786774	17002.633400	250.000000	113.450000	6.000000	1493.000000	4.000000	5.200000
std	4.238459	1.149150	6.965819	15875.651932	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
min	0.000000	0.000000	35.000000	290.000000	250.000000	113.450000	6.000000	1493.000000	4.000000	5.200000
25%	2.600000	0.800000	39.000000	7788.000000	250.000000	113.450000	6.000000	1493.000000	4.000000	5.200000
50%	7.200000	1.800000	44.000000	8794.000000	250.000000	113.450000	6.000000	1493.000000	4.000000	5.200000
75%	10.900000	2.600000	49.000000	27003.000000	250.000000	113.450000	6.000000	1493.000000	4.000000	5.200000
max	13.500000	20.000000	72.000000	73430.000000	250.000000	113.450000	6.000000	1493.000000	4.000000	5.200000

length	width	gross_weight	ncap_rating	claim_status	clusters
14018.000000	14018.000000	14018.000000	14018.000000	14018.000000	14018.000000
4300.000000	1790.000000	1720.000000	3.000000	0.064275	2.000000
0.000000	0.000000	0.000000	0.000000	0.245250	0.000000
4300.000000	1790.000000	1720.000000	3.000000	0.000000	2.000000
4300.000000	1790.000000	1720.000000	3.000000	0.000000	2.000000
4300.000000	1790.000000	1720.000000	3.000000	0.000000	2.000000
4300.000000	1790.000000	1720.000000	3.000000	0.000000	2.000000
4300.000000	1790.000000	1720.000000	3.000000	1.000000	2.000000

Nous constatons déjà certaines différences au niveau des caractéristiques. Visualisons les histogrammes et les diagrammes à bande:

Comapaison graphique des histogrammes



Comapaison graphique des diagramme en bande

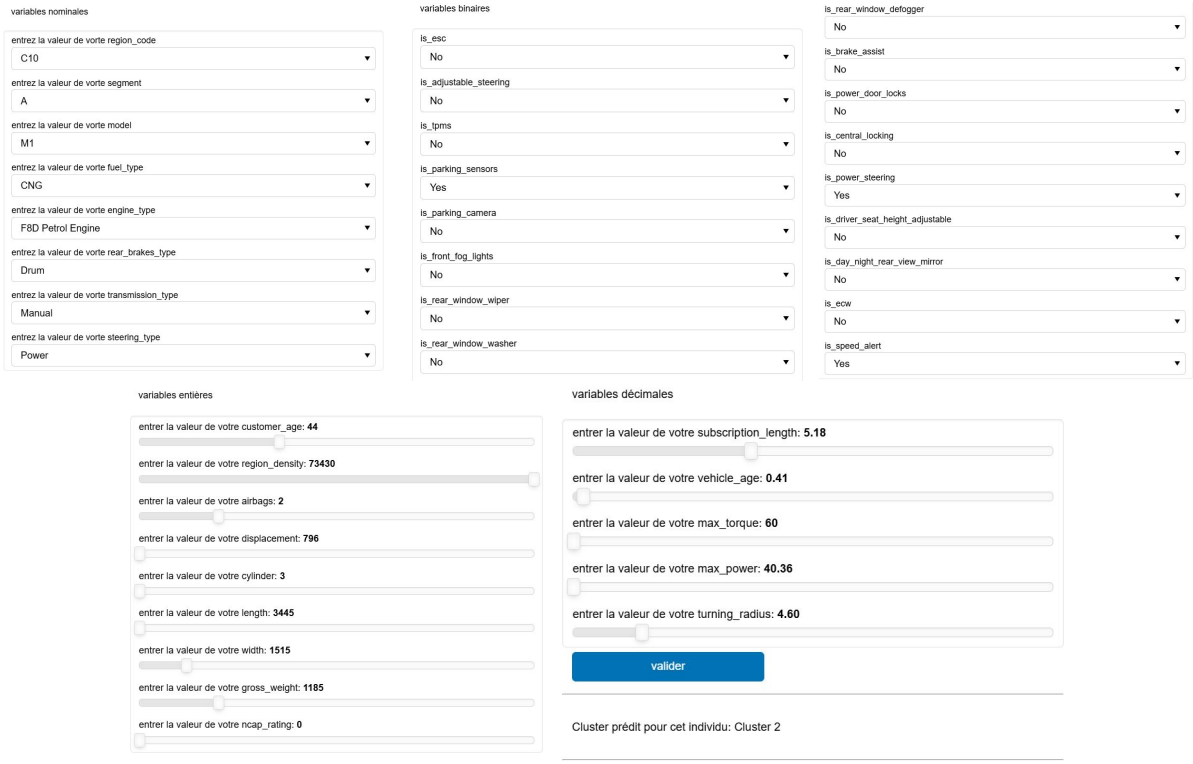


On constate que chaque cluster a en effet, ses propres particularités:

- pour la variable **segment**, Le cluster 0 a plus les individus des types 'C1' et 'B2' tandis que le cluster 1 a ceux des types 'A','B1','Utility'. le cluster 2 quant à lui a ceux de type 'C2'
- pour la variable **Model**, Le cluster 0 a majoritairement les individus dont le modèle est 'M6' ,tandis que le cluster 1 a majoritairement ceux des types 'M1'. le cluster 2 quant à lui a ceux de type 'M4'
- pour la variable '**max_torque**', le cluster 0 a majoritairement des valeurs autour de 120, le clusteur 1 autour de 60 , le cluster 2 est autour de 250
- pour la variable '**max_power**', le cluster 0 a majoritairement des valeurs autour de 90, le clusteur 1 autour de 40, le cluster 2 est autour de 113
- plusieurs autres différences peuvent être visualisé grâce aux graphes

4.2.2 classification d'un nouvel individu dans un des clusters

Pour positionner un nouvel individu dans l'un des clusters, nous utilisons encore l'interface précédemment présenté , en changeant l'objectif. Nous allons prendre un cas particulier et en faire la démonstration:



On voit que cet individu appartient au deuxième cluster(celui en orange)

4.2.3 Conclusion Partielle

Dans cette section consacrée aux méthodes d'apprentissage non supervisé, nous avons appliqué deux techniques complémentaires : l' Analyse en Composantes Principales (ACP) pour la réduction de dimension, puis l'algorithme de K-means pour la segmentation des observations.

- L' ACP a permis de projeter notre base de données dans un espace de plus faible dimension tout en conservant une part très élevée de l'information initiale. Les deux premières composantes expliquent à elles seules plus de 65% de la variance. L'interprétation du cercle de corrélation a mis en évidence les groupes de variables fortement corrélées ainsi que celles contribuant le plus à la formation des axes principaux.
- Cette réduction de dimension a ensuite facilité l'application de K-means, dont le nombre optimal de clusters a été déterminé grâce au coude et à la silhouette, tous deux suggérant $K = 3$. La classification obtenue a révélé trois groupes d'individus présentant des caractéristiques distinctes
- L'analyse comparative des trois clusters montre que chaque groupe correspond à un profil de véhicule et de comportement particulier, confirmant la capacité du

clustering à révéler des structures latentes dans les données. Cette partie met ainsi en évidence la puissance des approches non supervisées pour segmenter, comprendre, et explorer une base complexe sans variable cible.

5 COMPARAISON ENTRE LES DEUX TYPES DE MODÈLES

Les deux méthodes du Machine Learning étudiés ,supervisé et non supervisé, répondent à des besoins différents et se complètent dans une démarche analytique complète.

DOMAINE DE COMPARAISON	SUPERVISÉ	NON SUPERVISÉ
OBJET	apprend une fonction grâce à une variable cible connue. L'objectif est de prédire une classe ou une valeur.	**aucune cible n'est fournie ; l'objectif est de révéler des structures cachées dans les données.**
Utilité dans notre cas	indispensable pour prédire si un client fera une réclamation, en tenant compte du déséquilibre entre classes.	**utile pour comprendre des profils types de véhicules ou de clients sans dépendre de la variable cible, ce qui apporte plus d'informations**
interprétation	basé sur les métriques (accuracy, recall, F1-score...)	**basé sur la structure géométrique, la variabilité, et la cohésion interne des clusters.**
Complémentarité	répond à la question " que peut-on prédire ? "	**répond à la question " que peut-on comprendre ou segmenter ? "**

Les modèles supervisés ont montré que le déséquilibre marqué entre les classes constituait un défi majeur. Les techniques de rééchantillonnage, en particulier SMOTE, ont significativement amélioré les performances des modèles linéaires. Toutefois, c'est l'algorithme XGBoost qui s'est révélé le plus performant, grâce à sa capacité à capturer des interactions complexes et à gérer efficacement les données structurées. Il constitue ainsi une solution robuste pour la prédiction des réclamations d'assurance.

La seconde partie, dédiée aux modèles non supervisés, a mis en lumière la richesse informative de la base via l' ACP et le clustering. La réduction de dimension a facilité la compréhension des relations entre variables, tandis que K-means a identifié trois clusters distincts, révélant des profils de véhicules nettement différenciés.

6 CONCLUSION

Ce projet a permis d'explorer en profondeur l'ensemble du processus de data mining appliqué à une base de données d'assurance automobile. Après une phase rigoureuse de nettoyage, d'analyse exploratoire et de préparation des variables, deux grandes familles de modèles ont été mises en œuvre : les **modèles supervisés** et les **modèles non supervisés**.

En définitive, ce travail montre qu'une analyse combinée supervisée / non supervisée apporte une vision globale : la capacité à prédire des comportements, mais aussi à comprendre la structure interne de la population étudiée. Cette complémentarité est essentielle dans des domaines tels que l'assurance, où la prédiction des risques et la segmentation des clients sont deux enjeux stratégiques majeurs.

References

[1] **La revue IA** ,
Comment gérer le déséquilibre des classes (imbalanced dataset) en machine learning ?
https://larevueia.fr/comment-gerer-le-\desequilibre-des-\
classes-en-machine-learning/

[2] **linedata** ,
Principaux algorithmes d'apprentissage non supervisé
https://fr.linedata.com/principaux-algorithmes-d\
apprentissage-non-supervise

[3] **La revue IA** ,
Introduction au gradient boosting
https://larevueia.fr/algorithmes-de-gradient-boosting\
-et-introduction-a-xgboost/

[4] *Les arbres de décision (decision trees)*, *Christine Decaestecker, ULB Marco Saerens, UCL*
https://helios2.mi.parisdescartes.fr/~lomn/Cours/DM/Material/
ComplementsCours/decisiontree.pdf

[5] **Everlaab** ,
Arbre de décision : Le guide complet (+3 exemples utiles et concrets)
https://everlaab.com/arbre-de-decision/

[6] **Kaggle** ,
https://www.kaggle.com/code/litvinenko630/
decoding-insurance-claims-dynamics-with-data#
-Oversampling-the-Minority-Class

[7] **Kaggle** ,
https://www.kaggle.com/datasets/litvinenko630/insurance-claims